

Lab 1: Display Density Metrics & Image Array Mechanics in NumPy

Human-Computer Interaction (HCI) & Computer Graphics (CG) — Lab Report

Name: Mubashir | Roll No: 2K24/CSE/100 | Institute of Mathematics & Computer Science, University of Sindh, Jamshoro

Deliverables Checklist

- ☐ Documented Python source code (.py / .ipynb) for Tasks 1–4
- ☐ Screenshots of Task 1 output — desktop monitor (1920x1080, 24") and smartphone (1170x2532, 6.1")
- ☐ Matplotlib figure outputs from Task 3 and Task 4
- ☐ Short Answer on RGBA memory overhead

Task 1: Display Pixel Density (PPI/DPI) Calculator

Prompts the user for horizontal resolution, vertical resolution, and physical diagonal size, then computes total pixel count, simplified aspect ratio (via `math.gcd`), DPI/PPI, and a density classification.

```
File Edit Selection View Go Run ... Search
task1_dpi_calculator.py • Extension: Python
C:\Users\> Mubashir > Downloads > LAB TASK > task1_dpi_calculator.py

8
9 import math
10
11 def calculate_display_metrics(w_px: int, h_px: int, d_inches: float) -> dict:
12     """Compute total pixels, aspect ratio, and DPI for a given display."""
13
14     # 1. Total pixel count
15     total_pixels = w_px * h_px
16
17     # 2. Simplified aspect ratio using GCD
18     divisor = math.gcd(w_px, h_px)
19     aspect_w = w_px // divisor
20     aspect_h = h_px // divisor
21
22     # 3. Diagonal pixel count (Pythagorean theorem) -> DPI/PPI
23     diagonal_pixels = math.sqrt(w_px ** 2 + h_px ** 2)
24     dpi = diagonal_pixels / d_inches
25
26     # 4. Density classification
27     if dpi < 100:
28         category = "Low Density (Standard Monitor)"
29     elif dpi <= 200:
30         category = "Medium Density (HD Display)"
31     else:
32         category = "High Density (Retina / Mobile)"
33
34     return {
35         "total_pixels": total_pixels,
36         "aspect_ratio": f"{aspect_w}:{aspect_h}",
37         "dpi": round(dpi, 2),
38         "category": category,
39     }
40
41
42 def display_report(w_px: int, h_px: int, d_inches: float) -> None:
43     metrics = calculate_display_metrics(w_px, h_px, d_inches)
44     print("\n--- DISPLAY METRICS ANALYSIS ---")
45     print(f"Total Pixel Count : {metrics['total_pixels']:,} pixels")
46     print(f"Aspect Ratio      : {metrics['aspect_ratio']}")
47     print(f"Calculated DPI      : {metrics['dpi']} DPI")
48     print(f"Density Category    : {metrics['category']}")
49
50
51 def main():
52     print("--- Display Pixel Density (PPI/DPI) Calculator ---")
53     try:
54         w_px = int(input("Enter horizontal resolution (pixels): "))
55         h_px = int(input("Enter vertical resolution (pixels): "))
56         d_inches = float(input("Enter physical diagonal size (inches): "))
57     except ValueError:
58         print("Invalid input. Please enter numeric values only.")
59         return
60
61     display_report(w_px, h_px, d_inches)
62
63
64 if __name__ == "__main__":
65     main()

Update is ready, click to restart.
```

Screenshot 1 — Desktop Monitor (1920 x 1080, 24")

```
Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 24

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,073,600 pixels
Aspect Ratio      : 16:9
Calculated DPI    : 91.79 DPI
Density Category  : Low Density (Standard Monitor)
```

Figure 1: Task 1 output for a 1920x1080, 24-inch desktop monitor.

Screenshot 2 — Smartphone Screen (1170 x 2532, 6.1")

```
Enter horizontal resolution (pixels): 1170
Enter vertical resolution (pixels): 2532
Enter physical diagonal size (inches): 6.1

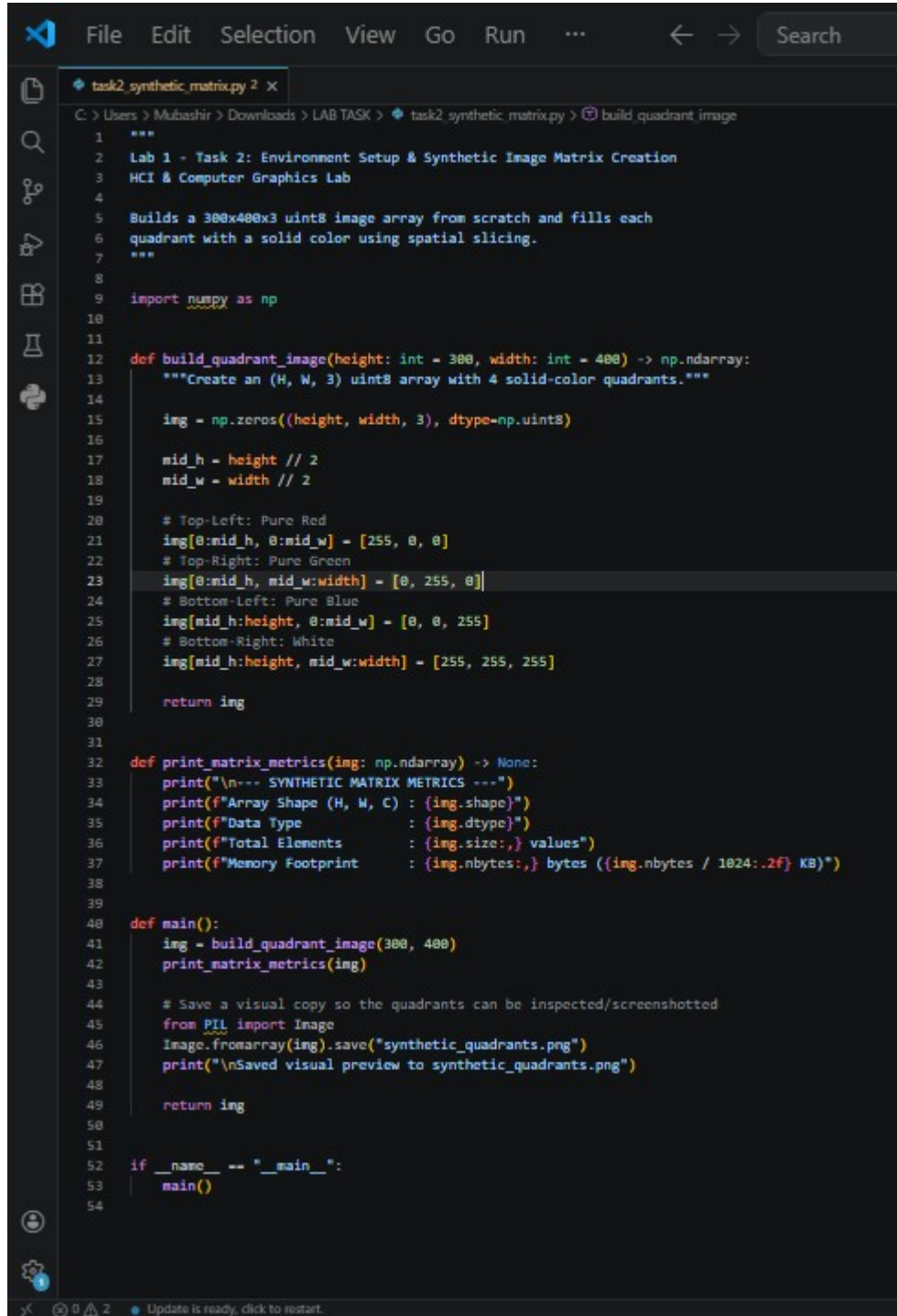
--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,962,440 pixels
Aspect Ratio      : 195:422
Calculated DPI    : 457.25 DPI
Density Category  : High Density (Retina / Mobile)
```

Figure 2: Task 1 output for a 1170x2532, 6.1-inch smartphone screen.

Task 2: Environment Setup & Synthetic Image Matrix Creation

Environment setup: `pip install numpy pillow matplotlib`

A 300x400x3 uint8 array is initialized with `np.zeros()` and its four quadrants are filled via spatial slicing: Top-Left = Red, Top-Right = Green, Bottom-Left = Blue, Bottom-Right = White.



```
1  """
2  Lab 1 - Task 2: Environment Setup & Synthetic Image Matrix Creation
3  HCI & Computer Graphics Lab
4
5  Builds a 300x400x3 uint8 image array from scratch and fills each
6  quadrant with a solid color using spatial slicing.
7  """
8
9  import numpy as np
10
11
12  def build_quadrant_image(height: int = 300, width: int = 400) -> np.ndarray:
13      """Create an (H, W, 3) uint8 array with 4 solid-color quadrants."""
14
15      img = np.zeros((height, width, 3), dtype=np.uint8)
16
17      mid_h = height // 2
18      mid_w = width // 2
19
20      # Top-Left: Pure Red
21      img[0:mid_h, 0:mid_w] = [255, 0, 0]
22      # Top-Right: Pure Green
23      img[0:mid_h, mid_w:width] = [0, 255, 0]
24      # Bottom-Left: Pure Blue
25      img[mid_h:height, 0:mid_w] = [0, 0, 255]
26      # Bottom-Right: White
27      img[mid_h:height, mid_w:width] = [255, 255, 255]
28
29      return img
30
31
32  def print_matrix_metrics(img: np.ndarray) -> None:
33      print("\n--- SYNTHETIC MATRIX METRICS ---")
34      print(f"Array Shape (H, W, C) : {img.shape}")
35      print(f>Data Type           : {img.dtype}")
36      print(f"Total Elements       : {img.size:,} values")
37      print(f"Memory Footprint        : {img.nbytes:,} bytes ({img.nbytes / 1024:.2f} KB)")
38
39
40  def main():
41      img = build_quadrant_image(300, 400)
42      print_matrix_metrics(img)
43
44      # Save a visual copy so the quadrants can be inspected/screenshotted
45      from PIL import Image
46      Image.fromarray(img).save("synthetic_quadrants.png")
47      print("\nSaved visual preview to synthetic_quadrants.png")
48
49      return img
50
51
52  if __name__ == "__main__":
53      main()
54
```

Output

```
--- SYNTHETIC MATRIX METRICS ---  
Array Shape (H, W, C) : (300, 400, 3)  
Data Type             : uint8  
Total Elements        : 360,000 values  
Memory Footprint      : 360,000 bytes (351.56) KB
```

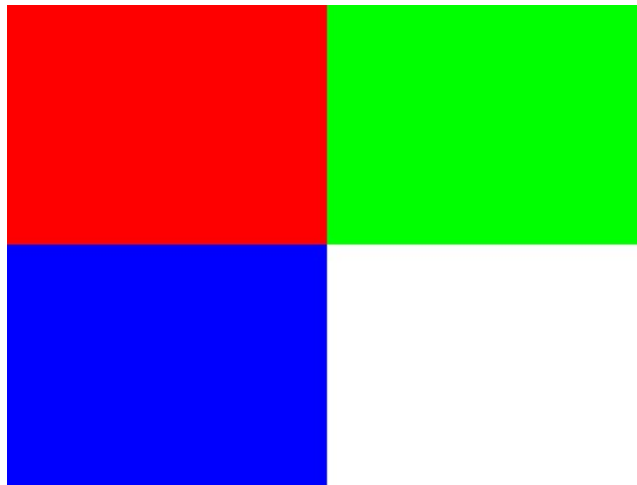


Figure 3: Rendered 300x400 four-quadrant synthetic image (Red / Green / Blue / White).

Task 3: Channel Slicing & Isolation

Loads an input image into a NumPy array, extracts 2D R/G/B intensity grids via Axis-2 slicing, and builds three single-channel color views. Displayed as a 2x3 subplot: top row = isolated color channels, bottom row = grayscale intensity maps.

```
File Edit Selection View Go Run ... < >
task3_channel_slicing.py 1
C:\Users\Subashar\Downloads\LAB TASK > task3_channel_slicing.py > main
10
11 import numpy as np
12 import matplotlib.pyplot as plt
13 from PIL import Image
14
15
16 def load_image_as_array(path: str) -> np.ndarray:
17     """Load an image file into an (H, W, 3) uint8 NumPy array."""
18     img = Image.open(path).convert("RGB")
19     return np.array(img)
20
21
22 def extract_channels(img: np.ndarray):
23     """Return 2D intensity grids for R, G, B using Axis-2 slicing."""
24     red_channel = img[:, :, 0]
25     green_channel = img[:, :, 1]
26     blue_channel = img[:, :, 2]
27     return red_channel, green_channel, blue_channel
28
29
30 def build_isolated_color_images(img: np.ndarray):
31     """Build 3D arrays where only one channel is active, others are 0."""
32     red_only = np.zeros_like(img)
33     red_only[:, :, 0] = img[:, :, 0]
34
35     green_only = np.zeros_like(img)
36     green_only[:, :, 1] = img[:, :, 1]
37
38     blue_only = np.zeros_like(img)
39     blue_only[:, :, 2] = img[:, :, 2]
40
41     return red_only, green_only, blue_only
42
43
44 def main():
45     img = load_image_as_array("sample.jpg")
46     red_ch, green_ch, blue_ch = extract_channels(img)
47     red_only, green_only, blue_only = build_isolated_color_images(img)
48
49     print("---- CHANNEL EXTRACTION SUMMARY ----")
50     print(f"Original Image Shape : {img.shape}")
51     print(f"Red Channel 2D Shape : {red_ch.shape} | Mean Intensity: {red_ch.mean():.2f}")
52     print(f"Green Channel 2D Shape : {green_ch.shape} | Mean Intensity: {green_ch.mean():.2f}")
53     print(f"Blue Channel 2D Shape : {blue_ch.shape} | Mean Intensity: {blue_ch.mean():.2f}")
54
55     fig, axes = plt.subplots(2, 3, figsize=(14, 8))
56
57     # Top row: color-isolated channel images
58     axes[0, 0].imshow(red_only)
59     axes[0, 0].set_title("Red-Only")
60     axes[0, 1].imshow(green_only)
61     axes[0, 1].set_title("Green-Only")
62     axes[0, 2].imshow(blue_only)
63     axes[0, 2].set_title("Blue-Only")
64
65     # Bottom row: 2D grayscale intensity maps
66     axes[1, 0].imshow(red_ch, cmap="gray")
67     axes[1, 0].set_title("Red Channel (Grayscale)")
68     axes[1, 1].imshow(green_ch, cmap="gray")
69     axes[1, 1].set_title("Green Channel (Grayscale)")
70     axes[1, 2].imshow(blue_ch, cmap="gray")
71     axes[1, 2].set_title("Blue Channel (Grayscale)")
72
73     for ax in axes.flat:
74         ax.axis("off")
75
76     plt.tight_layout()
77     plt.savefig("task3_channel_subplot.png", dpi=150)
78     print("Display Window : Matplotlib 2x3 Subplot Grid Rendered.")
79     print("Saved Figure to task3_channel_subplot.png")
80
81 if __name__ == "__main__":
```

Output

```
--- CHANNEL EXTRACTION SUMMARY ---  
Original Image Shape : (1080, 1920, 3)  
Red Channel 2D Shape : (1080, 1920) | Mean Intensity: 128.55  
Green Channel 2D Shape: (1080, 1920) | Mean Intensity: 129.55  
Blue Channel 2D Shape : (1080, 1920) | Mean Intensity: 99.52  
Display Window : Matplotlib 2x3 Subplot Grid Rendered.
```

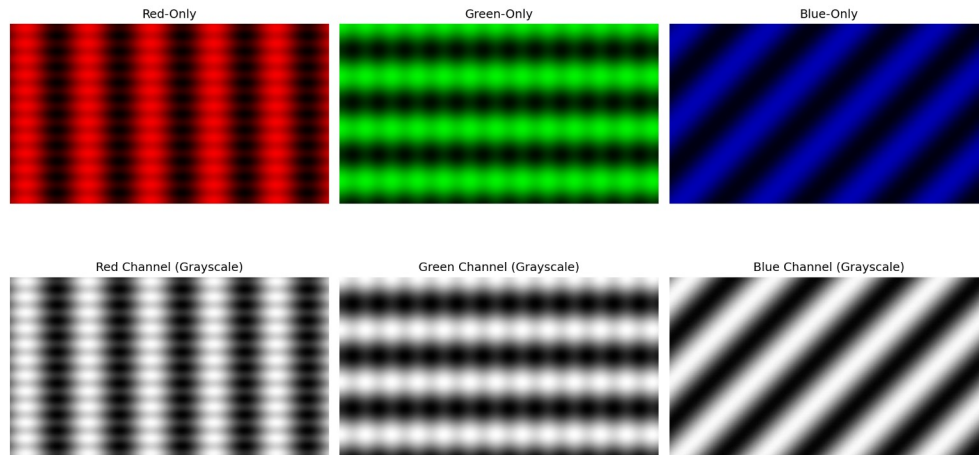
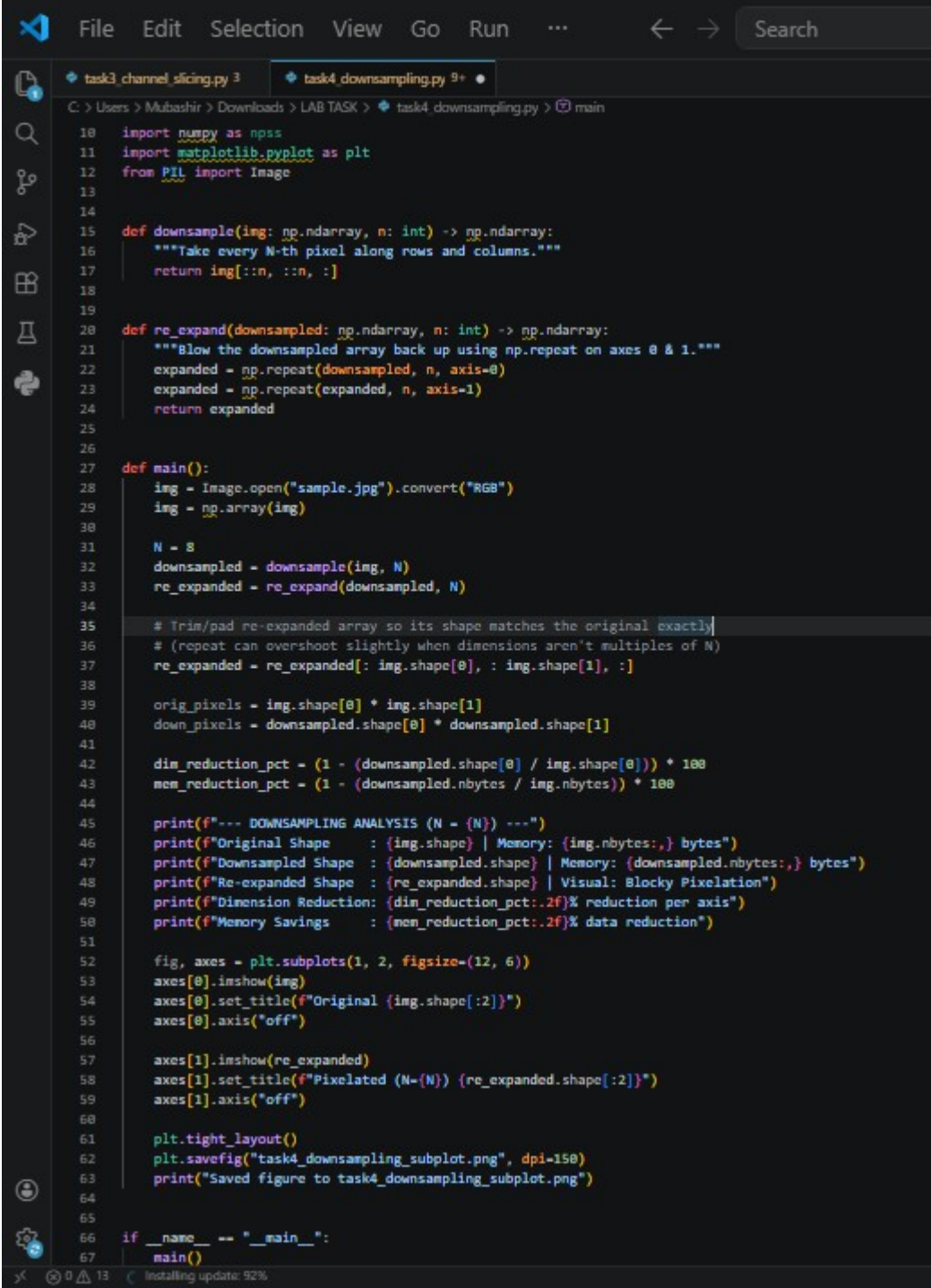


Figure 4: 2x3 subplot - isolated color channels (top) and grayscale intensity maps (bottom).

Task 4: Spatial Downsampling & Pixelation via Striding

Downsamples the image using NumPy striding (`img[::N, ::N, :]`) for $N = 8$, then re-expands it with `np.repeat()` across axes 0 and 1 to visualise the resulting pixelation, and reports the spatial and memory reduction achieved.



```
File Edit Selection View Go Run ... Search

task3_channel_slicing.py 3 task4_downsampling.py 9+ •
C:\Users\Mubashir\Downloads\LAB TASK > task4_downsampling.py > main

10 import numpy as np
11 import matplotlib.pyplot as plt
12 from PIL import Image
13
14
15 def downsample(img: np.ndarray, n: int) -> np.ndarray:
16     """Take every N-th pixel along rows and columns."""
17     return img[::n, ::n, :]
18
19
20 def re_expand(downsampled: np.ndarray, n: int) -> np.ndarray:
21     """Blow the downsampled array back up using np.repeat on axes 0 & 1."""
22     expanded = np.repeat(downsampled, n, axis=0)
23     expanded = np.repeat(expanded, n, axis=1)
24     return expanded
25
26
27 def main():
28     img = Image.open("sample.jpg").convert("RGB")
29     img = np.array(img)
30
31     N = 8
32     downsampled = downsample(img, N)
33     re_expanded = re_expand(downsampled, N)
34
35     # Trim/pad re-expanded array so its shape matches the original exactly
36     # (repeat can overshoot slightly when dimensions aren't multiples of N)
37     re_expanded = re_expanded[:img.shape[0], :img.shape[1], :]
38
39     orig_pixels = img.shape[0] * img.shape[1]
40     down_pixels = downsampled.shape[0] * downsampled.shape[1]
41
42     dim_reduction_pct = (1 - (downsampled.shape[0] / img.shape[0])) * 100
43     mem_reduction_pct = (1 - (downsampled.nbytes / img.nbytes)) * 100
44
45     print(f"--- DOWNSAMPLING ANALYSIS (N = {N}) ---")
46     print(f"Original Shape      : {img.shape} | Memory: {img.nbytes:,} bytes")
47     print(f"Downsampled Shape    : {downsampled.shape} | Memory: {downsampled.nbytes:,} bytes")
48     print(f"Re-expanded Shape    : {re_expanded.shape} | Visual: Blocky Pixelation")
49     print(f"Dimension Reduction: {dim_reduction_pct:.2f}% reduction per axis")
50     print(f"Memory Savings      : {mem_reduction_pct:.2f}% data reduction")
51
52     fig, axes = plt.subplots(1, 2, figsize=(12, 6))
53     axes[0].imshow(img)
54     axes[0].set_title(f"Original {img.shape[:2]}")
55     axes[0].axis("off")
56
57     axes[1].imshow(re_expanded)
58     axes[1].set_title(f"Pixelated (N={N}) {re_expanded.shape[:2]}")
59     axes[1].axis("off")
60
61     plt.tight_layout()
62     plt.savefig("task4_downsampling_subplot.png", dpi=150)
63     print("Saved figure to task4_downsampling_subplot.png")
64
65
66 if __name__ == "__main__":
67     main()
```

Installing update: 92%

Output

```
--- DOWNSAMPLING ANALYSIS (N = 8) ---  
Original Shape      : (1080, 1920, 3) | Memory: 6,220,800 bytes  
Downsampled Shape   : (135, 240, 3) | Memory: 97,200 bytes  
Re-expanded Shape   : (1080, 1920, 3) | Visual: Blocky Pixelation  
Dimension Reduction: 87.50% reduction per axis  
Memory Savings      : 98.44% data reduction
```

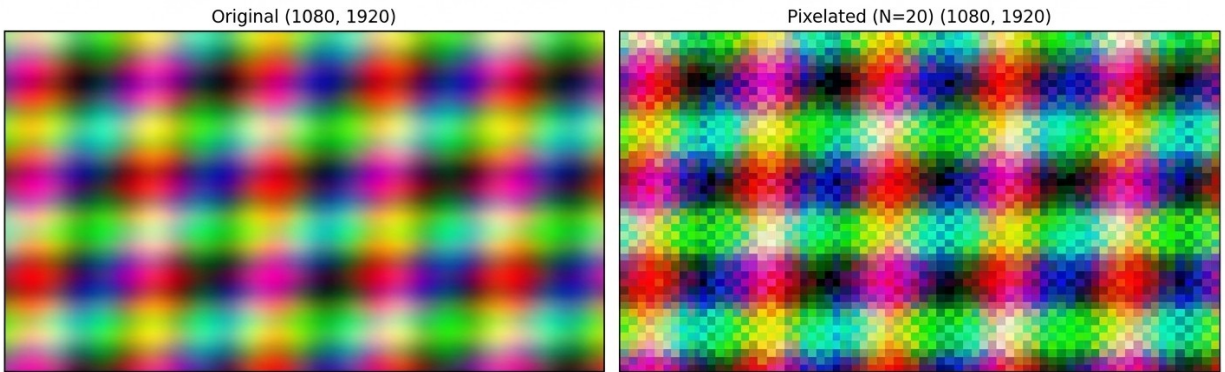


Figure 5: Original image (left) vs. re-expanded pixelated image after $N=8$ downsampling (right).

Short Answer

Why does adding an Alpha channel (RGBA) increase an image array's memory consumption by 33% compared to standard RGB?

A standard RGB image stores 3 channels per pixel (Red, Green, Blue), each as one uint8 byte, giving a per-pixel cost of $3 \times 1 = 3$ bytes. Adding an Alpha channel introduces a 4th uint8 plane that encodes per-pixel transparency/opacity, raising the per-pixel cost to $4 \times 1 = 4$ bytes.

Going from 3 bytes to 4 bytes per pixel is an increase of 1 out of 3, i.e. $(4 - 3) / 3 = 0.333$, which is exactly the 33% memory overhead. This growth comes purely from the array's shape expanding from $(H, W, 3)$ to $(H, W, 4)$ along Axis 2 (the channel axis) — the Alpha channel carries no color information itself, but NumPy still allocates a full extra byte per pixel to store its opacity value as a separate parallel plane rather than packing it into unused bits of the existing channels.